

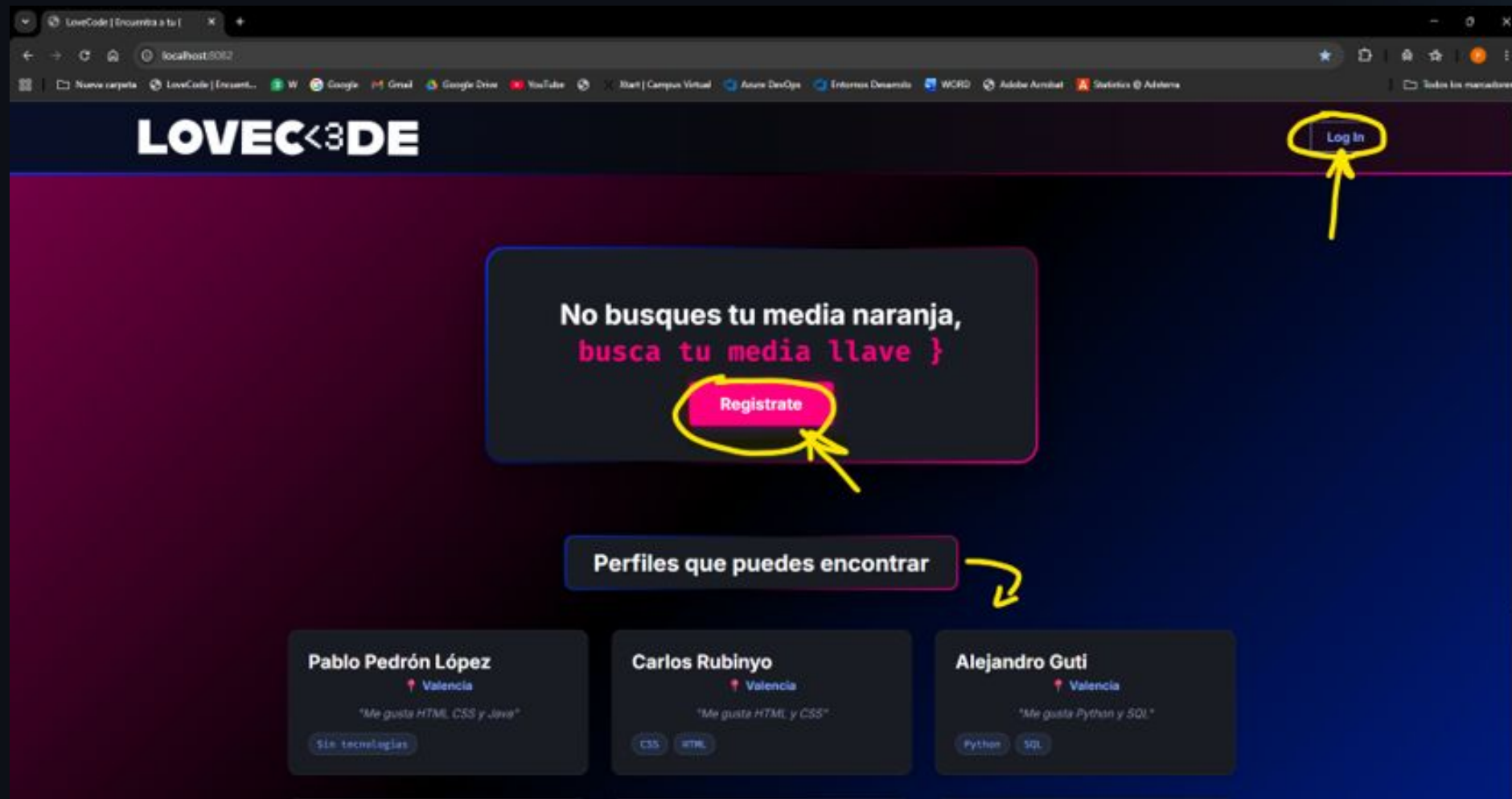


LOVEC<3DE

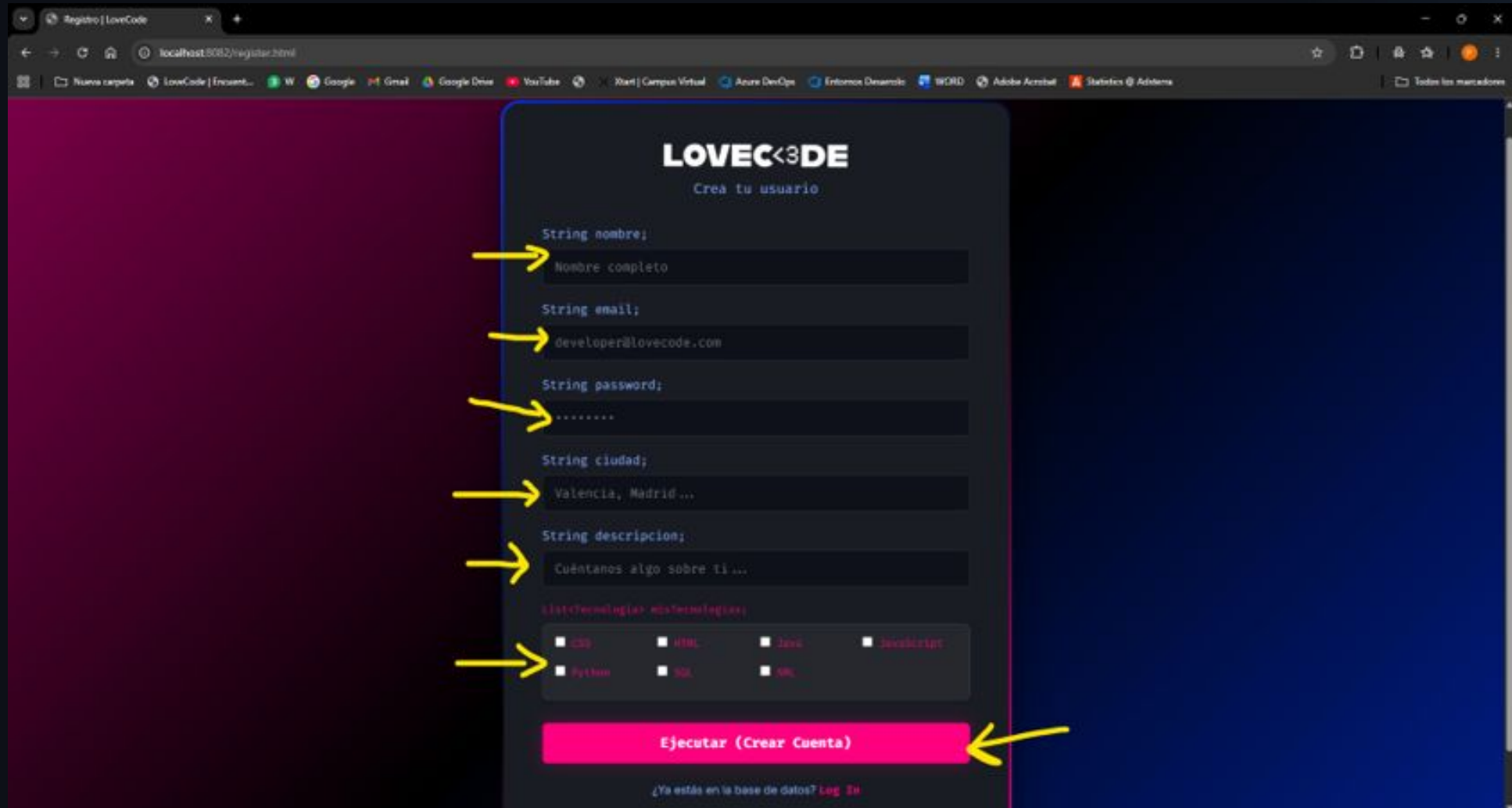
Defensa de Proyecto Final · 1o DAW · 2025-26

Pablo Pedrón López

QUE ES LOVECODE



QUE ES LOVECODE

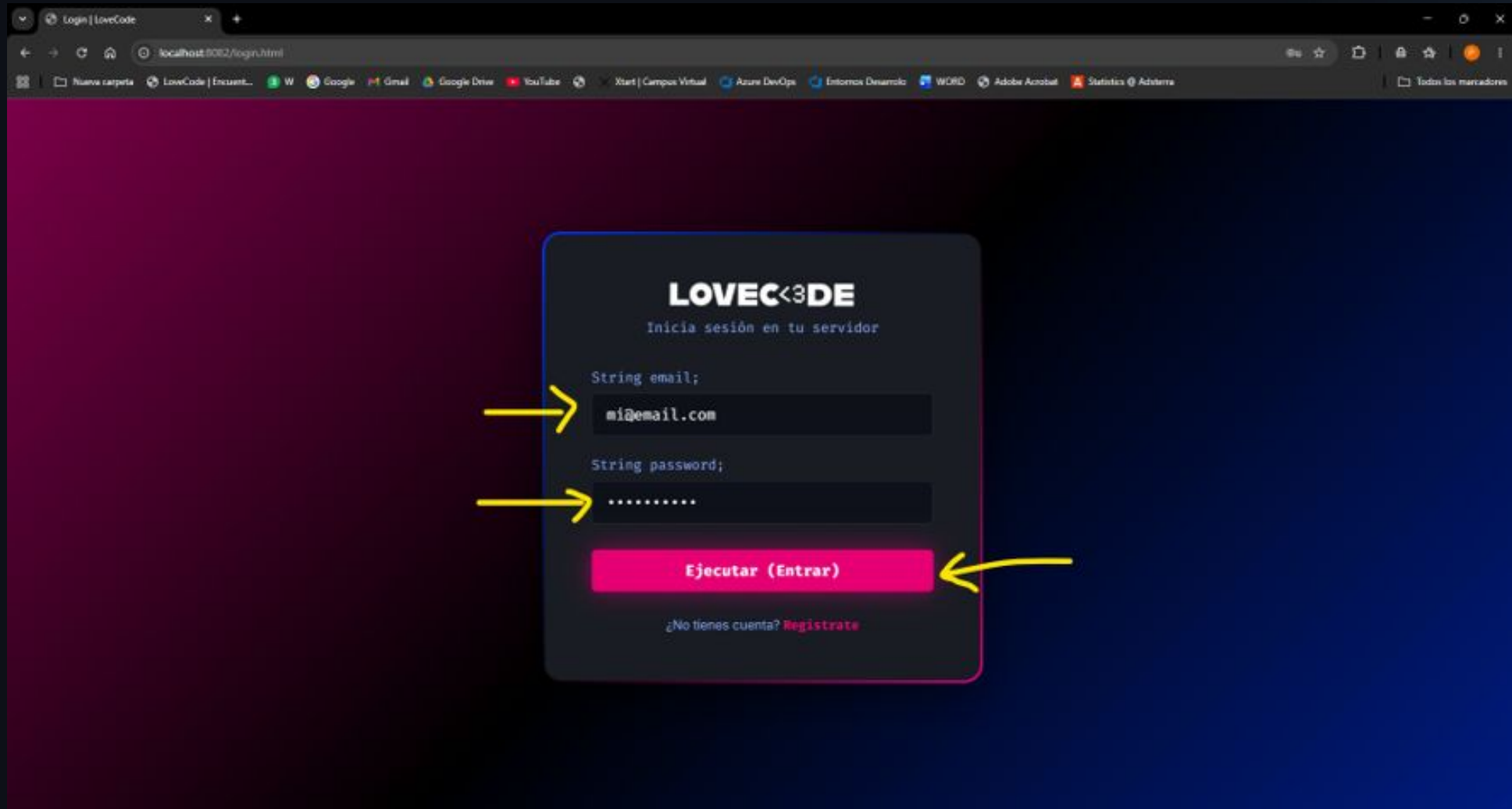


The screenshot shows a web browser window with the URL `localhost:8082/register.html`. The page features a registration form for 'LOVECODE' with the subtitle 'Crea tu usuario'. The form includes several text input fields, a checkbox section for technologies, and a submit button. Yellow arrows point to each of these elements:

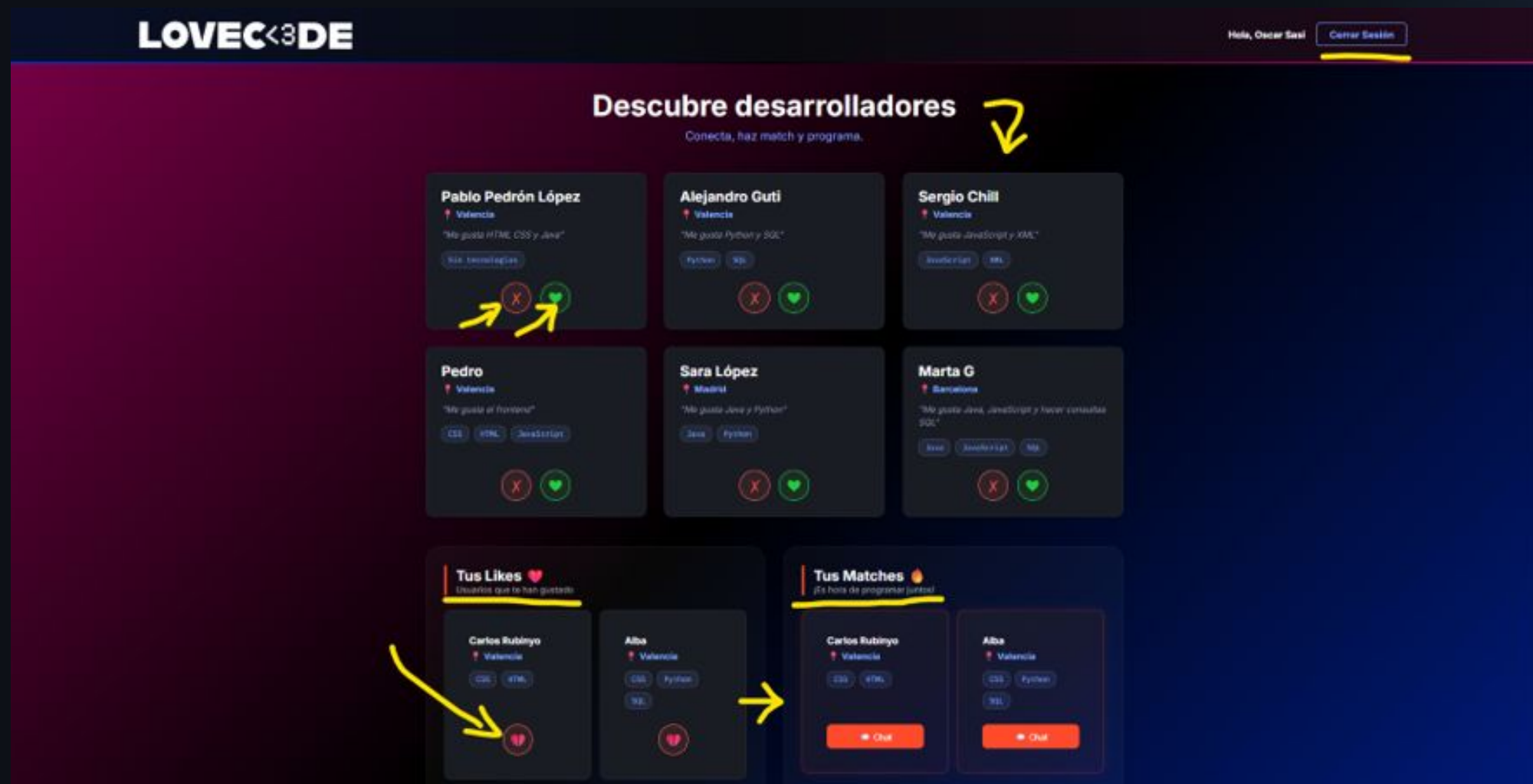
- `String nombre;` points to the 'Nombre completo' input field.
- `String email;` points to the 'developer@lovecode.com' input field.
- `String password;` points to the password input field, which shows six dots.
- `String ciudad;` points to the 'Valencia, Madrid...' input field.
- `String descripcion;` points to the 'Cuéntanos algo sobre ti...' input field.
- A yellow arrow points to the checkbox section titled `¿Listo/tecnologías que te gustan?`, which contains checkboxes for C#, HTML, Java, Javascript, Python, SQL, and PHP.
- A yellow arrow points to the 'Ejecutar (Crear Cuenta)' button.

At the bottom of the form, there is a link: `¿Ya estás en la base de datos? Log In`.

QUE ES LOVECODE



QUE ES LOVECODE



| Arquitectura del Sistema



FRONTEND

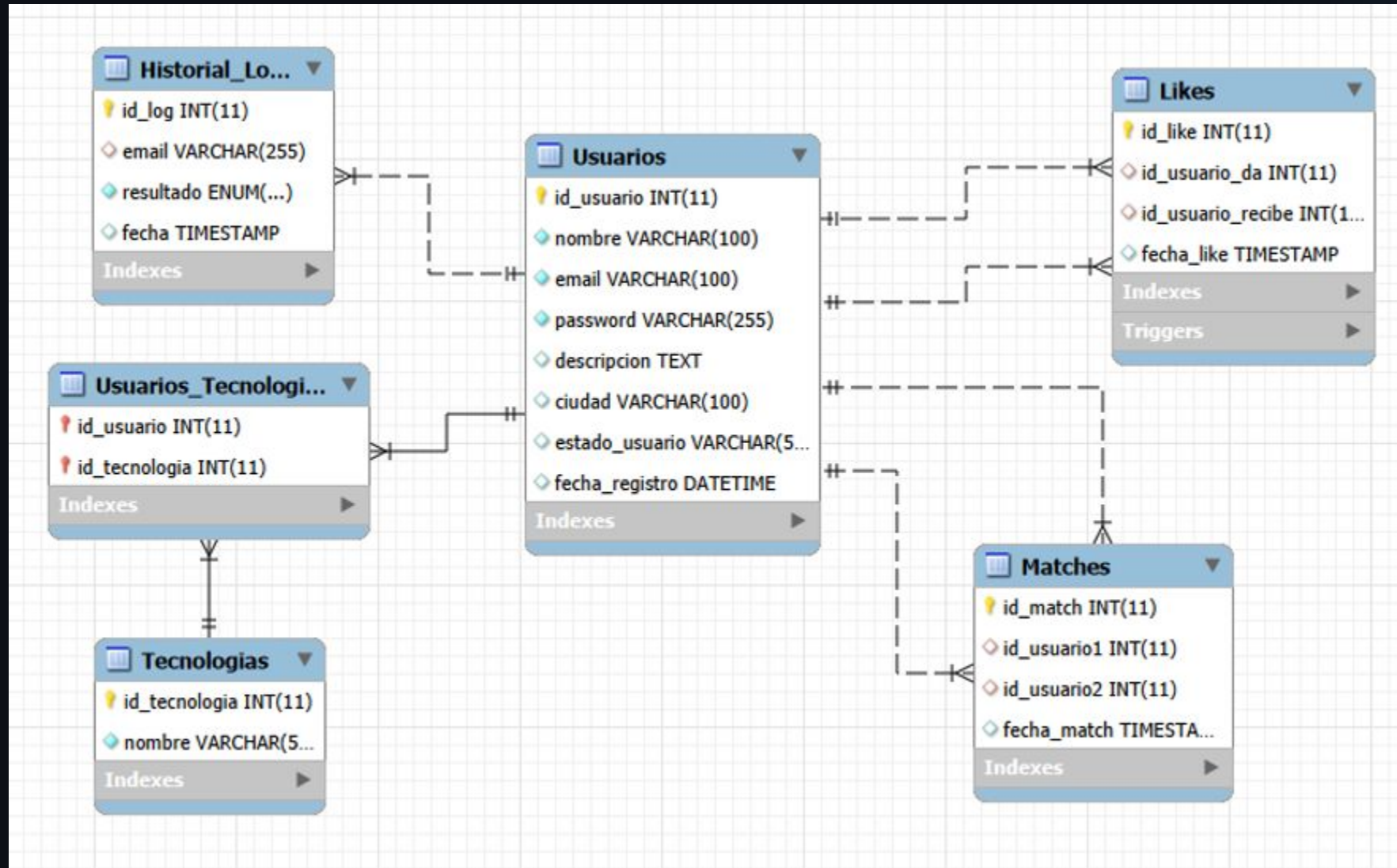


BACKEND



BBDD (DEBIAN)

DIAGRAMA E/R



ELEMENTOS DESTACADOS AL CREAR BBDD



UNIQUE KEY

Garantiza la **unicidad absoluta** en campos clave como **email** y **tecnologías** de forma nativa, bloqueando registros duplicados antes de llegar al backend.



ON DELETE CASCADE

Mantiene la **integridad referencial** automatizada en interacciones (**Likes** y **Matches**). Si un usuario es eliminado, sus relaciones se purgan limpiamente en cascada.



ON DELETE SET NULL

Estrategia de **preservación estadística** aplicada a **Historial_Logs**. Si una cuenta se elimina, el histórico de inicios de sesión se conserva asignando **NULL** al email.



RESTRECCIÓN ENUM

Establece un **dominio cerrado y rígido** en el resultado de auditorías. El campo solo acepta de manera nativa los valores predefinidos **EXITO** o **FALLO**.

PROCEDIMIENTOS ALMACENADOS

Routines (SP)

```
CREATE PROCEDURE sp_registrar_usuario(  
    IN p_nombre VARCHAR(100),  
    IN p_email VARCHAR(100),  
    ...  
)  
BEGIN  
    INSERT INTO Usuarios ...  
END;
```



Automatización mediante Triggers



tr_after_insert_like

Comprueba reciprocidad y tecnologías comunes tras cada Like. Si hay coincidencia, inserta el **Match** automáticamente.



tr_after_delete_like

Mantiene la integridad eliminando el **Match** asociado si un usuario deshace su interés.

La base de datos **toma decisiones** por sí misma.

Los 3 Tipos de Backup Implementados



1. Manual (GUI)

Gestionado visualmente desde **MySQL Workbench**.



2. Terminal interactivo

Volcado lógico mediante utilidades nativas en consola (mariadb-dump).



3. Script Automatizado

Orquestado mediante un script en **PowerShell** que conecta via SSH a Debian, extrae los datos y programa la tarea de forma diaria en Windows.

| USUARIOS

lovecode LECTOR



lector

192.168.204.129:3306

lovecode ADMINISTRADOR



administrador

192.168.204.129:3306

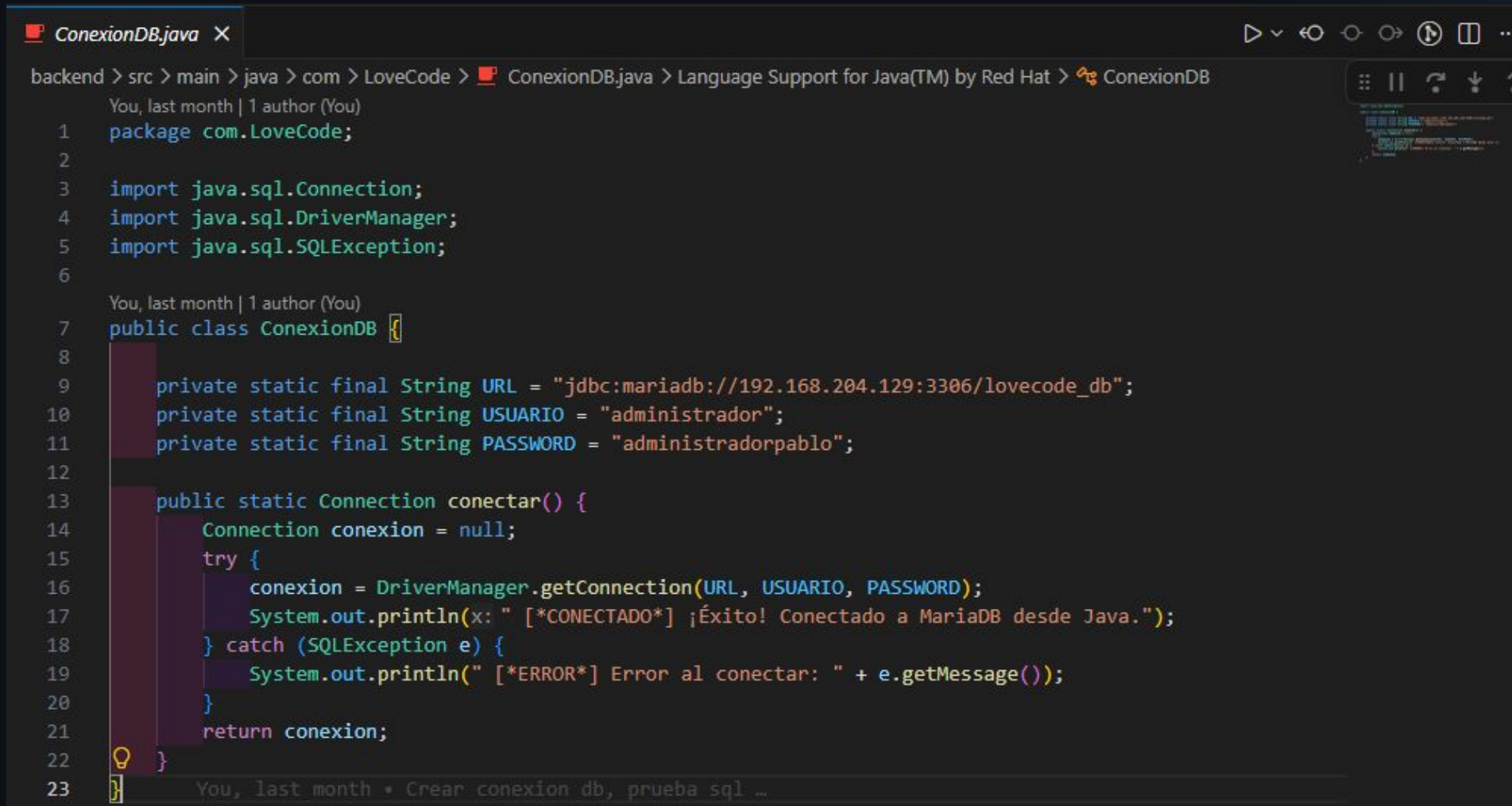
lovecode DESARROLADOR



desarrollador

192.168.204.129:3306

CONEXIONADO JDBC



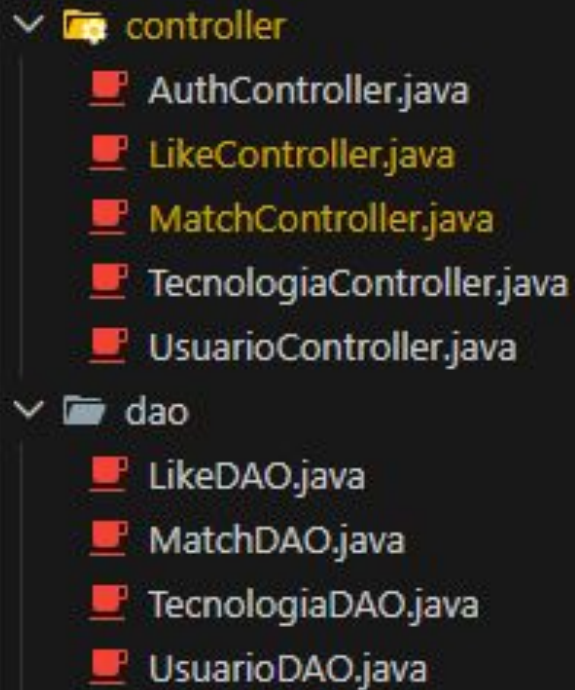
```
ConexionDB.java X
backend > src > main > java > com > LoveCode > ConexionDB.java > Language Support for Java(TM) by Red Hat > ConexionDB
You, last month | 1 author (You)
1 package com.LoveCode;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6
7 You, last month | 1 author (You)
8 public class ConexionDB {
9     private static final String URL = "jdbc:mariadb://192.168.204.129:3306/lovecode_db";
10    private static final String USUARIO = "administrador";
11    private static final String PASSWORD = "administradorpablo";
12
13    public static Connection conectar() {
14        Connection conexion = null;
15        try {
16            conexion = DriverManager.getConnection(URL, USUARIO, PASSWORD);
17            System.out.println(x: " [*CONECTADO*] ¡Éxito! Conectado a MariaDB desde Java.");
18        } catch (SQLException e) {
19            System.out.println(" [*ERROR*] Error al conectar: " + e.getMessage());
20        }
21        return conexion;
22    }
23 }
```

MAVEN

The screenshot displays an IDE interface with a dark theme. On the left, the 'EXPLORER' sidebar shows a project structure for 'LOVECODE'. The 'backend' directory is expanded, revealing subdirectories like '.github', '.vscode', 'database', and 'src'. The 'src' directory is further expanded, showing 'main' > 'java' > 'com' > 'LoveCode'. Under 'LoveCode', there are 'config' (with 'SecurityConfig.java'), 'controller' (with 'AuthController.java', 'LikeController.java', 'MatchController.java', 'TecnologiaController.java', and 'UsuarioController.java'), and 'dao' (with 'LikeDAO.java', 'MatchDAO.java', 'TecnologiaDAO.java', 'UsuarioDAO.java', 'ConexionDB.java', 'Main.java', and 'Usuario.java').

The main editor area shows the 'pom.xml' file for the 'backend' project. The breadcrumb navigation at the top of the editor reads 'backend > pom.xml > project > dependencies'. The XML content is as follows:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd"
5         <modelVersion>4.0.0</modelVersion>
6
7         <parent>
8             <groupId>org.springframework.boot</groupId>
9             <artifactId>spring-boot-starter-parent</artifactId>
10            <version>3.2.4</version>
11            <relativePath/> <!-- lookup parent from repository -->
12        </parent>
13
14        <groupId>com.LoveCode</groupId>
15        <artifactId>backend</artifactId>
16        <version>1.0-SNAPSHOT</version>
17
18        <properties>
19            <java.version>21</java.version>
20        </properties>
21
22        <dependencies>
23            <dependency>
24                <groupId>org.springframework.boot</groupId>
25                <artifactId>spring-boot-starter-web</artifactId>
26            </dependency>
27
```



Controlador	Método + Ruta	Operación
AuthController	POST /api/auth/registro	Registra un nuevo usuario: hashea la contraseña con BCrypt, llama a sp_registrar_usuario y asigna tecnologías.
AuthController	POST /api/auth/login	Autentica al usuario: busca por email, compara hash BCrypt y registra el intento en Historial_Logs.
UsuarioController	GET /api/usuarios	Devuelve todos los usuarios activos. Si se pasa ?idUser=X, filtra los ya vistos/likeados para el dashboard.
LikeController	POST /api/likes	Registra un like. Tras la inserción, consulta si el trigger generó un match y lo informa en la respuesta.
LikeController	DELETE /api/likes	Elimina un like. El trigger tr_after_delete_like borra el match correspondiente automáticamente.
LikeController	GET /api/likes/dados/{id}	Devuelve la lista de usuarios a los que el usuario ha dado like.
MatchController	GET /api/matches/{id}	Devuelve la lista de matches del usuario logueado.
TecnologiaController	GET /api/tecnologias	Devuelve el catálogo de tecnologías disponibles para el formulario de registro.

TESTS UNITARIOS

```
You, 5 days ago | 1 author (You)
public class UsuarioTest {

    // Prueba 1: Validar email correcto
    @Test
    public void testEmailValido() {
        // Arrange
        Usuario usuario = new Usuario();
        String emailValido = "pablo@example.com";

        // Act
        usuario.setEmail(emailValido);

        // Assert - verificar que el email cumple el formato básico
        assertTrue(usuario.getEmail().contains(s: "@"), "El email debe contener @");
        assertTrue(usuario.getEmail().contains(s: "."), "El email debe contener un punto");
        assertFalse(usuario.getEmail().startsWith(prefix: "@"), "El email no puede empezar con @");
    }
}
```


| Frontend



HTML5



CSS3



JavaScript

Peticiones Asíncronas

Comunicación Asíncrona (Fetch API)

- 🔗 **Estructura Async/Await:** Manejo secuencial y limpio de promesas en llamadas al backend, reduciendo la complejidad del código.
- 🛡️ **Resolución de CORS:** Intercambio exitoso de datos JSON gracias al mapeo de cabeceras de origen cruzado en las peticiones.
- ⚡ **Actualización Dinámica:** Interceptación inmediata de interacciones (Likes/Matches) para actualizar las vistas sin recargas del navegador.

```
async function ejecutarLogin(event) {
  event.preventDefault();

  const creds = {
    email: document.getElementById('login-email').value,
    password: document.getElementById('login-pass').value
  };

  try {
    const res = await fetch(`${URL_BASE}/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(creds)
    });

    const info = await res.json();

    if (res.ok) {
      alert("Bienvenido, " + info.nombre);
      localStorage.setItem('lovecode_user', info.nombre);
      localStorage.setItem('lovecode_user_id', info.id); // Guardamos el ID para los likes
      window.location.href = "dashboard.html";
    } else {
      alert("Acceso denegado: " + info.error);
    }
  } catch (error) {
    console.error("Error:", error);
  }
}
```

Control de Versiones y Gestión

 ppedron1 / LoveCode

Q Type / to search

<> Code Issues Pull requests 1 Agents Actions Projects Security and quality Insights Settings

 LoveCode Public

Unpin Watch 0 Fork 0 Starred 1

main 23 Branches 0 Tags

Go to file T Add file <> Code

 ppedron1 Merge pull request #38 from ppedron1/historial_logs c792ee1 · 4 days ago 82 Commits

backend	Ahora se registran los logs correctamente	4 days ago
documentación	Se sube documentacion actual	last month
frontend	añadir funcionalidad	2 weeks ago
.gitignore	Update .gitignore	3 weeks ago
README.md	Add project structure details to README.md	last month

About

LoveCode

- Readme
- Activity
- 1 star
- 0 watching
- 0 forks

Releases

No releases published

[Create a new release](#)

¡Gracias!

¿Alguna pregunta?

Pablo Pedrón López

Repositorio:

github.com/ppedron1/lovecode
